

# HackLISP-0

A simple programming language for the Hack computer

Ethan Charoenpitaks

2026-05-12

HackLISP-0

# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32 |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33 |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

# The problem

- Open-source software is built on other open-source operating systems
- Thus, we must trust that none of the open-source OSes we've used in the past 30 years is malicious
- Instead, let's try to bootstrap!
  - Start from a small binary seed
  - After that, only ever input human-readable source code
  - Eventually, get a modern OS
- Past efforts
  - x86\_64 bootstrapping effort – stage0, live-bootstrap, GNU Mes, GNU Guile
    - Actually successful!
    - Very long bootstrap chain
    - x86\_64 processors are not trustable (Management Engine, very complex)

# Hack computer

- We made this last semester!
- As you know, understandable by high schoolers
  - ...at least compared to what's in your computers...
- Can build from scratch!
  - Small enough to be made out of vacuum tubes

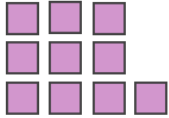
## Hack assembly, Jack VM, Jack?

- So hard to parse!
- I don't want to do Projects 6, 7, 8, 10, 11 in assembly...
- Let's find a simpler language

## SectorLISP

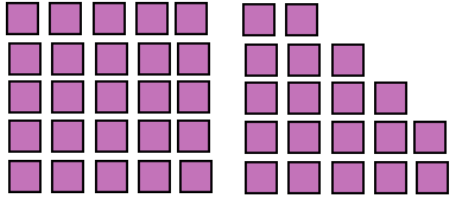
- Minimal LISP that fits in 436 bytes of i8086 machine code
- No integers, no direct memory access
- But small, yet you can write readable code!

# Binary Footprint Comparison



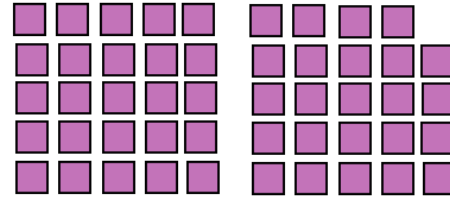
Brainf#\*k

Not a true language but we built it anyway for comparison  
99 bytes



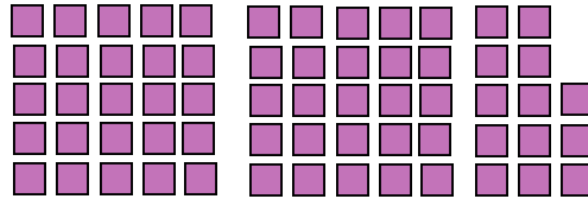
SectorLISP

Tiniest Programming Language  
436 bytes w/ garbage collect



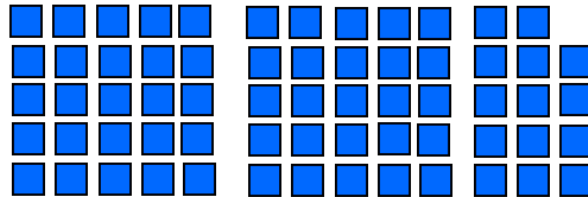
SectorFORTH & BootBASIC

The Former Champions  
491 & 510 bytes



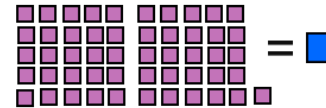
L06183 Leuconostoc Phage L5

Tiniest Organic Creature  
613 bytes



LISP 1.5

IBM 7090 LISP c. 1961  
32 kilobytes<sup>[7]</sup>





## What I did

- HackLISP-0 – basic language
- Essentially an extension of SectorLISP
  - better documented
  - better-named keywords
- Has integers and arithmetic
- Can read and write memory
  - If we combine Hack's RAM and ROM, then we can use this to make a compiler
- Written in C/Jack polyglot
  - Too many things to figure out, I'm also lazy

# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32 |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33 |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

## Cons cell

A cons cell is a pair, represented in text as an **s-expression**

( *head* . *tail* ).

Both *head* and *tail* are stored as pointers, to objects of any data type.

A cons cell evaluates to the result of calling its head with its tail as the arguments.

## List

A list is a list of things. It is represented as a cons cell, whose *tail* is another cons cell. The last element of the list is represented by ( *head* . NIL ). For example:

$$(A\ B\ C\ D\ E) = (A . (B . (C . (D . (E . NIL))))))$$

A list evaluates to the result of calling its head with its tail as the arguments.

## Atom

An *atom* is a basic thing, it's not made out of any other Lisp objects.

## Symbol

A symbol is stored as a pointer to a null-terminated C string on the atom stack. It can represent whatever you want. A symbol evaluates to whatever value is bound to it. The program will crash if you evaluate a symbol that's not bound to it.

As an exception, **NIL** and **TRUE** evaluate to themselves. **()** and **NIL** are indistinguishable.

## Integer

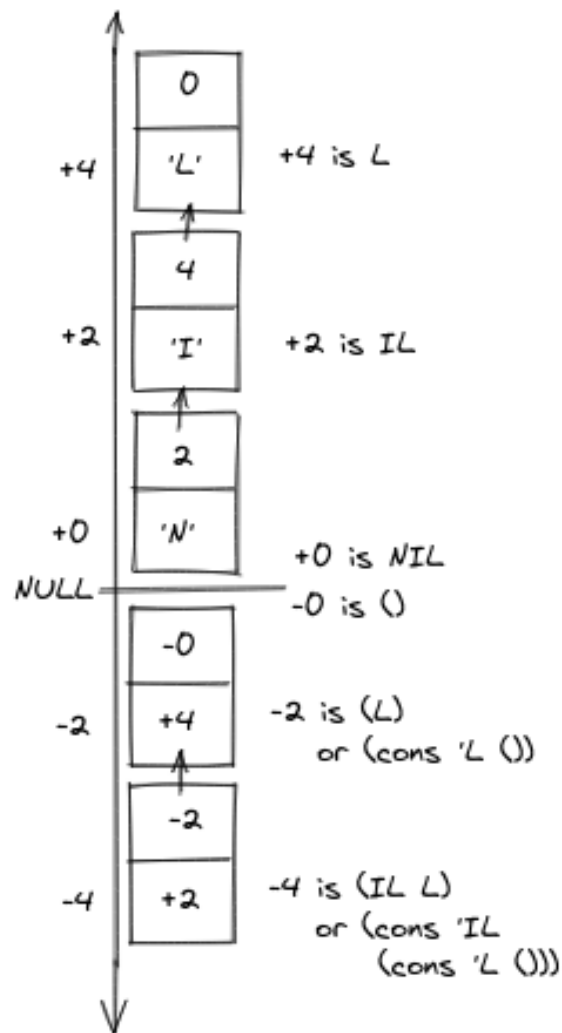
An integer represents, well, an integer. It evaluates to itself.

It's stored as three integers in memory. The first is **-1**, to make it clear it's not a symbol. The second is the actual value. The third is **0** to make it a null-terminated string.

# Atom

## Data storage

- Atom stack grows up from **NIL** (in our case 9312)
- Pair stack grows down
- Atom stack holds null-terminated C strings and integers.
- Image on the right – an old version of SectorLISP
  - Unlike the image we use null-terminated C string instead of LISP list for the strings





# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | *                          | 32 |
| The problem .....       | 3  | cond .....           | 19 | /                          | 33 |
| Hack computer .....     | 4  | = .....              | 20 | &                          | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 |                            | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming       | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | +                    | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | —                    | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

## quote

(**quote**  $x$ ) returns  $x$ , without evaluating it.

```
> (quote a)
```

```
a
```

```
> (quote (a b c))
```

```
(a b c)
```

**Important:** **quote** is the only builtin function which does not evaluate its arguments.

## cond

**(cond**  $(p_1 e_1) \dots (p_n e_n)$ )

The  $p_k$  are evaluated in order until one returns **TRUE**. When that happens, the value of  $e_k$  is returned as the value of the whole expression.

```
> (cond (NIL (quote no)) (TRUE (quote yes))))
```

```
yes
```

```
> (cond (TRUE (quote first)) (TRUE (quote second))))
```

```
first
```

=

(=  $x$   $y$ ) returns **TRUE** if  $x$  and  $y$  are both atoms and are the same, or if they are lists stored in the same memory position. Otherwise, it returns **NIL**.

```
> (= (quote a) (quote a))
```

```
TRUE
```

```
> (= (quote b) (quote a))
```

```
NIL
```

## cons

(**cons**  $x$   $y$ ) returns  $(x . y)$ .

```
> (cons (quote a) (quote (b c)))
```

```
(a b c)
```

```
> (cons (quote a) (quote b))
```

```
(a . b)
```

## atom?

(**atom?**  $x$ ) returns **TRUE** if  $x$  is an atom, otherwise it returns **NIL**.

```
> (atom? (quote (a b)))
```

```
NIL
```

```
> (atom? 3)
```

```
TRUE
```

```
> (atom? (quote atom?))
```

```
TRUE
```

## head

(**head**  $x$ ) returns the head of  $x$  (its first element).

```
> (head (quote (a b)))
```

```
a
```

Most Lisps call it **car** (Contents of the Address Register).



## tail

(**head**  $x$ ) returns the tail of  $x$ :

- If it's a cons cell, its second element
- If it's a list, everything after the first element

```
> (tail (quote (a b c)))
```

```
(b c)
```

```
> (tail (quote (a . b)))
```

```
b
```

Most Lisps call it **cdr** (Contents of the Decrement Register).

# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32 |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33 |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

## **read**

(**read**) waits for the user to enter in an s-expression, then it parses the s-expression and returns it.

## **print**

(**print**  $x$ ) prints the value  $x$  (as an  $s$ -expression) to the screen.

(**print**) with no arguments makes a new line.

It returns the arguments.

## **integer?**

**(integer?  $x$ )** returns **TRUE** if  $x$  is an integer and **NIL** if not.

+

(+  $x$   $y$ ) returns  $x + y$ .

—

$(- x)$  returns  $-x$ .

$(- x y)$  returns  $x - y$ .



`*`

`(* x y)` returns  $x * y$ .

/

(/ x y) returns  $\frac{x}{y}$ .

**&**

**(&  $x$   $y$ )** returns the bitwise AND of  $x$  and  $y$ .

|

(|  $x$   $y$ ) returns the bitwise OR of  $x$  and  $y$ .

## **mem-get**

(**mem-get**  $x$ ) returns the value at  $\text{RAM}[x]$ .

## **mem-put!**

(**mem-put!**  $x$   $y$ ) sets  $\text{RAM}[x]$  to  $y$ .

# Outline

|                         |    |                      |    |                            |           |
|-------------------------|----|----------------------|----|----------------------------|-----------|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32        |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33        |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34        |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35        |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36        |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37        |
| What I did .....        | 8  | tail .....           | 24 | <b>Lambdas</b> .....       | <b>38</b> |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40        |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42        |
| List .....              | 12 | print .....          | 28 | Imperative                 |           |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43        |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44        |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46        |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |



# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32 |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33 |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

## Recursion and naming

```
((lambda (a b) (a 1))
```

```
; Note that they don't actually have to start with lambda-[name]
```

```
; Since we don't actually check for the lambda keyword we can do this
```

```
(quote (lambda-a (x) (cons x (b x))
```

```
(quote (lambda-b (x) (cons x (a x))
```

```
)
```

# Imperative programming

```
((lambda (main prog) (main))
  (quote (lambda-main () (prog
    (print (quote (Hello world)))
    (print)
    (print (quote (Goodbye)))
    (print)
  )))
; It pretends to take no arguments,
; but it actually takes infinitely many
(quote (lambda-prog () NIL))
)
```

# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | *                          | 32 |
| The problem .....       | 3  | cond .....           | 19 | /                          | 33 |
| Hack computer .....     | 4  | = .....              | 20 | &                          | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 |                            | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

## Boilerplate

```
((lambda (main nil? evcon evlis lookup pairlis eval apply prog)
  (main main (quote (NIL . NIL) (TRUE . TRUE)))
)
 (quote (lambda-main (main builtins) (prog
  (print (quote HL0>))
  (print (eval (read)))
  (print)
  (main main builtins)
)))
 (quote (lambda-nil? (x)
  (= x NIL)
))
 (quote (lambda-prog (args...) NIL))
```

## pairlis

```
; "Zips" two lists (x and y) together, and appends a.
;
; Example:
;   pairlis[(A B C); (U V W); ((D . X) (E . Y))] =
;     ((A . U) (B . V) (C . W) (D . X) (E . Y))
;
; From LISP 1.5 manual:
;   This function gives the list of pairs of corresponding elements of
;   the lists x and y, and appends this to the list a.
(quote (lambda-pairlis (x y a)
  (cond
    ((nil? x) a)
    (TRUE (cons (cons (head x) (head y)) (pairlis (tail x) (tail y) a)))
  )
))
```



# lookup

```
; Searches an alist for a given key.  
; Returns the value for said key.  
;  
; Called "assoc" in some implementations.  
(quote (lambda-lookup (key alist)  
  (cond  
    ((= key (head (head alist))) (tail (head alist)))  
    (TRUE (lookup key (tail alist)))  
  )  
))
```

## eval

```
; Evaluates the expression e in the environment env.
; env is an alist (association list) -- a list of (key . value) pairs
(quote (lambda-eval (e env)
  (cond
    ((nil? e) e)
    ((integer? e) e)
    ((atom? e) (lookup e env))

    ; If e is a list (quote ____), return _____.
    ; We use head(tail(e)) because it's actually (quote . (____ . NIL))
    ((= (head e) (quote quote)) (head (tail e)))

    ; If e is a conditional, punt to evcon to evaluate it.
    ((= (head e) (quote cond)) (evcon (tail e) env))

    ; If the above fails,
```

```
; head(e) is the first item (so the function name)
; use evlis to evaluate the rest of the items
; call apply to apply the function against the values
(TRUE (apply (head e) (evlis (tail e) env) env))
)
))
```

## evlis

```
; Loops through each expression in exprs and evaluates it.  
; Returns a list of the evaluated expressions.  
;  
; Examples:  
; (evlis '(a b c) '((a . 1)(b . 2)(c . 3))) -> '(1 2 3)  
(quote (lambda-evlis (exprs env)  
  (cond  
    ((nil? exprs) NIL)  
    (TRUE (cons (eval (head exprs) env) (evlis (tail exprs) env)))  
  )  
))
```

## apply

; Applies the function f to the argument x, in the environment env.

```
(quote (lambda-apply (f x env)
  (cond
    ((atom? f) (cond
      ((= f (quote =)) (= (head x) (head (tail x))))
      ((= f (quote cons)) (cons (head x) (head (tail x))))
      ((= f (quote integer?)) (integer? (head x)))
      ((= f (quote atom?)) (atom? (head x)))
      ((= f (quote head)) (head (head x)))
      ((= f (quote tail)) (tail (head x)))
      ((= f (quote read)) (read))
      ((= f (quote print)) (print (head x)))
      ((= f (quote +)) (+ (head x) (head (tail x))))
      ((= f (quote 0)) (cond
        ((nil? (tail x)) (- (head x)))
        (TRUE (- (head x) (head (tail x))))
      ))
    ))
```

```

))
((= f (quote *)) (* (head x) (head (tail x))))
((= f (quote /)) (/ (head x) (head (tail x))))
((= f (quote &)) (& (head x) (head (tail x))))
((= f (quote |)) (| (head x) (head (tail x))))
((= f (quote mem-get)) (mem-get (head x)))
((= f (quote mem-put!)) (mem-put! (head x) (head (tail x))))
; Otherwise assume f is a variable bound to some function or builtin.
(TRUE (apply (lookup f env) x env))
))

```

```

; If the first element is a list...
; We assume f is a lambda-expression of form
;   (lambda ARGS RETVAL)
; then run: (LISP 1.5)
;   eval[RETVAL; pairlis[ARGS; x; env]]
; i.e. we use pairlis to bind the args in the environment,

```

```
; then evaluate it.  
;  
; We skip checking that it starts with `lambda` to reduce the code size  
; and the runtime.  
(TRUE (eval (head (tail (tail f))) (pairlis (head (tail f)) x env)))  
)  
)
```

## evcon

```
; Evaluates a conditional expression c, in the environment env.  
; c is a list of (TEST BODY) [non-dotted] lists  
;   i.e. (TEST . (BODY . NIL))  
; env is, of course, an environment specified as an alist.  
;  
; evcon iterates in order through the elements of c and  
; evals the first expression of each pair. If it returns a nonnull  
; value, the body is evaluated and returned.
```

```
(quote (lambda-evcon (c env)  
  (cond  
    ((eval (head (head c)) env)  
      (eval (head (tail (head c))) env))  
    (TRUE  
      (evcon (tail c) env))  
  )  
))
```



# Outline

|                         |    |                      |    |                            |    |
|-------------------------|----|----------------------|----|----------------------------|----|
| Bootstrapping .....     | 1  | quote .....          | 18 | * .....                    | 32 |
| The problem .....       | 3  | cond .....           | 19 | / .....                    | 33 |
| Hack computer .....     | 4  | = .....              | 20 | & .....                    | 34 |
| Hack assembly, Jack VM, |    | cons .....           | 21 | .....                      | 35 |
| Jack? .....             | 5  | atom? .....          | 22 | mem-get .....              | 36 |
| SectorLISP .....        | 6  | head .....           | 23 | mem-put! .....             | 37 |
| What I did .....        | 8  | tail .....           | 24 | Lambdas .....              | 38 |
| Data types .....        | 9  | Extra builtins ..... | 25 | Useful tricks .....        | 40 |
| Cons cell .....         | 11 | read .....           | 27 | Recursion and naming .     | 42 |
| List .....              | 12 | print .....          | 28 | Imperative                 |    |
| Atom .....              | 13 | integer? .....       | 29 | programming .....          | 43 |
| Data storage .....      | 15 | + .....              | 30 | Metacircular interpreter . | 44 |
| Basic builtins .....    | 16 | — .....              | 31 | Boilerplate .....          | 46 |

|                      |    |
|----------------------|----|
| pairlis .....        | 47 |
| lookup .....         | 48 |
| eval .....           | 49 |
| evlis .....          | 51 |
| apply .....          | 52 |
| evcon .....          | 55 |
| Future steps .....   | 56 |
| Making HackLISP-0    |    |
| useful for           |    |
| bootstrapping .....  | 58 |
| HackLISP- $n$ .....  | 59 |
| Nicer syntaxes ..... | 61 |

# Making HackLISP-0 useful for bootstrapping

- Combine Hack's RAM and ROM
  - Then we can put code into RAM and jump to it
  - allowing a compiler!
- Basic text editor in assembly
  - Current HackLISP-0 limits the program to only 60 characters
- Write a HackLISP-0 interpreter in assembly
  - Daunting task...

## HackLISP-*n*

- Why didn't the x86\_64 compiler project use LISP?

LISP primitives defined in McCarthy's 1960 paper [Turns out he missed many essential things] with some modern improvements like Lexical scope, let expressions and raw string support. Turns out you need proper LISP macros in order to produce something useful in bootstrapping. Adding LISP macros in assembly simply is a task no one wants to do.

— Jeremiah Orians

- So we need to add these kinds of things!
  - Lexical scope
  - Let expressions
  - Raw strings
  - Macros

- We need a compiler
  - Otherwise it becomes slower, and slower, and slower...
  - and we'll run out of RAM!
- Garbage collection
- Mutable objects???
- Might be a bad idea
  - But we might run out of RAM otherwise
- Try out nicer syntaxes (next slide)
- Get to Scheme or Common Lisp
  - Opens up a huge world of useful programs we can run
  - If we get to Scheme, we can use GNU Mes, a C compiler!

# Nicer syntaxes

## Wisp

<https://www.draketo.de/software/wisp>

`display "Hello World!"`  $\mapsto$  `(display "Hello World!")`

`define : factorial n`  $\mapsto$  `(define (factorial n)`  
    `if : zero? n`  $\mapsto$  `(if (zero? n)`  
        `. 1`  $\mapsto$  `1`  
        `* n : factorial {n - 1}`  $\mapsto$  `(* n (factorial {n - 1})))`)

# Nicer syntaxes

## Sweet-expressions

<https://srfi.schemers.org/srfi-110/srfi-110.html>

```
define fibfast(n)      ; Typical function notation
  if {n < 2}            ; Indentation, infix {...}
    n                  ; Single expr = no new list
    fibup n 2 1 0      ; Simple function calls
```

```
(define (fibfast n)
  (if (< n 2)
      n
      (fibup n 2 1 0)))
```